Joins
(further reading)
Structured Query Language (SQL)

Joining more than two tables

Table Name	Description	Primary Key (PK)	Foreign Keys (FK)
Customers	Stores customer details	CustomerID	-
Orders	Stores orders placed by customers	OrderID	CustomerID, EmployeeID, ShipperID
Employees	Stores employees who process orders	EmployeeID	-
Shippers	Stores shipping company details	ShipperID	-

Following query retrieves only customers who have placed orders, along with employees and shipping details.

```
SELECT
    C.CustomerName,
    C.City,
    O.OrderID,
    O.Amount,
    O.OrderDate,
    E.EmployeeName AS ProcessedBy,
    S.ShipperName AS ShippedBy
FROM Orders O
INNER JOIN Customers C ON O.CustomerID = C.CustomerID
INNER JOIN Employees E ON O.EmployeeID = E.EmployeeID
INNER JOIN Shippers S ON O.ShipperID = S.ShipperID;
```

Real-Life Examples of SQL Joins in Business Applications

Joins are extremely common in real business database queries. Let's look at typical use cases for each type of join:

1. **Using INNER JOIN to retrieve customer orders:** Imagine an e-commerce database with a **Customers** table and an **Orders** table. To get a report of all orders along with the customer name who placed each order, you would use an inner join on the customer ID. For example:

   ```
   SELECT C.CustomerName, C.City, O.OrderID, O.OrderDate, O.TotalAmount
   FROM Customers C
   INNER JOIN Orders O
     ON C.CustomerID = O.CustomerID;
   ```

   This query will output only those orders that have a matching customer, which is typically what you want (every order should have a customer, and customers with no orders are not included in this list). In a business context, this could be used to list all sales with customer details for a given period, etc. The inner join is appropriate because you're inherently interested in records that exist in both tables (customers who made orders).

2. **Using LEFT JOIN to find customers with and without orders:** Now suppose management wants to see **which customers have never placed an order**. You can get this by left joining Customers to Orders and looking for NULLs in the Orders fields. For example:

```
SELECT C.CustomerName, O.OrderID
FROM Customers C
LEFT JOIN Orders O
  ON C.CustomerID = O.CustomerID
WHERE O.OrderID IS NULL;
```

This query would return customers who have no orders (because the left join includes all customers, and the WHERE O.OrderID IS NULL filters to those with no matching order). Even without the WHERE clause, a left join gives you a combined list of all customers and their orders (if any). A report might use a left join to show **all customers, highlighting those without orders** (NULL indicating no orders). This can help identify inactive customers. The left join is great for optional relationships – e.g., list all products and include sales data if present, etc.

3. **Using RIGHT JOIN to get all shipped orders and their customer details:** Right joins are less common, but consider a scenario with a **Shipments** table (all orders that have shipped) and you want to list all those shipments along with customer info. If there's a possibility some shipments are not linked to a customer (perhaps due to data issues or shipments generated from legacy data), a right join of Customers (left) to Shipments (right) would ensure **all shipments** appear. For instance:

   ```
   SELECT S.ShipmentID, S.TrackingNumber, C.CustomerName, C.Phone
   ```

```
FROM Customers C
RIGHT JOIN Shipments S
  ON C.CustomerID = S.CustomerID;
```

This yields every shipment, including those that might not have a corresponding customer (with the customer columns NULL in that case). In practice, you could usually achieve the same with a left join from Shipments to Customers. The key point is that a right join ensures no shipped order is left out of the results. It might be used in auditing to catch shipments that aren't tied to a known customer.

4. **Using FULL JOIN for comprehensive product and supplier reports:** Consider a company that has a **Products** table and a **Suppliers** table. They want a master list that shows all products along with their supplier, and also lists any suppliers who currently have no products. A full outer join can accomplish this in one query:

```
SELECT P.ProductName, P.Category, S.SupplierName, S.ContactInfo
FROM Products P
FULL JOIN Suppliers S
  ON P.SupplierID = S.SupplierID;
```

This will return every product (with its supplier info if there is a match) and every supplier (with product info if there is a match). Products without a supplier (maybe discontinued items or missing data) will show NULL for supplier fields, and suppliers with no products will show NULL for product fields. A full join provides a **complete overview** of the inventory and supplier base in one result. For example, it could feed a report that lists all suppliers and products, perhaps to plan outreach to suppliers who aren't supplying anything or to see products without an active supplier.

5. **Using CROSS JOIN for pricing combinations in an e-commerce setting:** Cross joins can be useful for generating combinations of values. For instance, an online store might want to display a table of all possible combinations of **size and color for a T-shirt**, or all combinations of a product and a discount percentage for a sale. Suppose you have a Sizes table (S - Small, Medium, Large) and a Colors table (C - Red, Blue, Green). A cross join query SELECT S.Size, C.Color FROM Sizes S CROSS JOIN Colors C; would produce every size-color pair. In a pricing scenario, if you had a **Discounts** table with various discount rates, you could cross join Products with Discounts to list all potential discounted prices for each product. This is helpful in planning or generating exhaustive option lists. However, in business apps, cross join is used sparingly and typically on small tables, because the output grows quickly with the size of the inputs.

6. **Using SELF JOIN for hierarchical employee-manager relationships:** Many organizations store employee information in a single table that includes a manager ID field. A self join is perfect for producing an **employee org chart report**. For example, if there is an Employees table with EmployeeID and ManagerID, you can write:

```
SELECT E.EmployeeName, E.Title, M.EmployeeName AS ManagerName, M.Title AS ManagerTitle
FROM Employees E
```

```
LEFT JOIN Employees M
  ON E.ManagerID = M.EmployeeID;
```

This will list each employee alongside their manager's name (with NULL for top-level bosses who have no manager). Real-life use: HR might use this query to list all employees with their direct manager, or to find cases where ManagerID is invalid (if the manager comes out NULL when it shouldn't). Similarly, self joins can handle other hierarchical data like category-subcategory relationships in a product catalog, where a Category table might self-reference a parent category.

These examples illustrate how each join type solves a particular kind of business query. Choosing the right join depends on the question being asked: are we only interested in strictly matching data (inner join), do we need a "master list" including those with no matches (outer joins), or do we need all combinations (cross join), or a recursive relationship (self join)?